

Taller de APIs (RESTful)

ComCom

Departamento de Computación
FCEyN UBA

Primer Cuatrimestre 2026



Introducción a las APIs

¿Cómo se comunican dos programas que no se conocen?

¹Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

²Claude, Codex, Gemini, etc.

¿Cómo se comunican dos programas que no se conocen?

Las API (Application Programming Interface) son mecanismos que permiten a dos **componentes de software comunicarse** entre sí mientras ambos sistemas **cumplan** con un **contrato/especificación**.

¹Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

²Claude, Codex, Gemini, etc.

¿Cómo se comunican dos programas que no se conocen?

Las API (Application Programming Interface) son mecanismos que permiten a dos **componentes de software comunicarse** entre sí mientras ambos sistemas **cumplan** con un **contrato/especificación**.

Ejemplos

- El lector de tarjetas cuando pagás en un local se comunica con la API de la pasarela de pagos.
- Tu IDE¹ favorito habla mediante una API con tu IA/Agente/LLM² de confianza.

¹Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

²Claude, Codex, Gemini, etc.

Existen varios tipos de APIs, algunas son:

- RPC
- WebSocket
- **RESTful**

Siendo estas últimas las más famosas, y las que nos interesarán en el transcurso del taller.

Tipos de APIs

Existen varios tipos de APIs, algunas son:

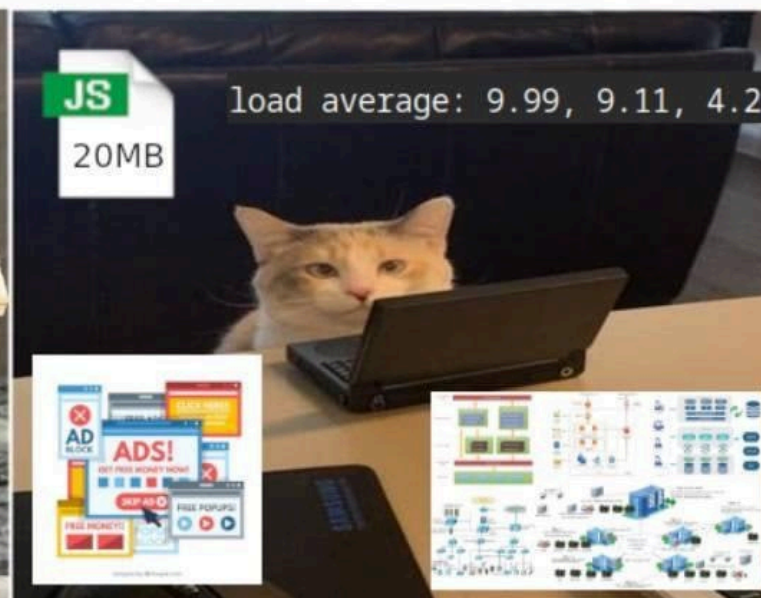
- RPC
- WebSocket
- **RESTful**

Siendo estas últimas las más famosas, y las que nos interesarán en el transcurso del taller.

**born to
write asm and C**



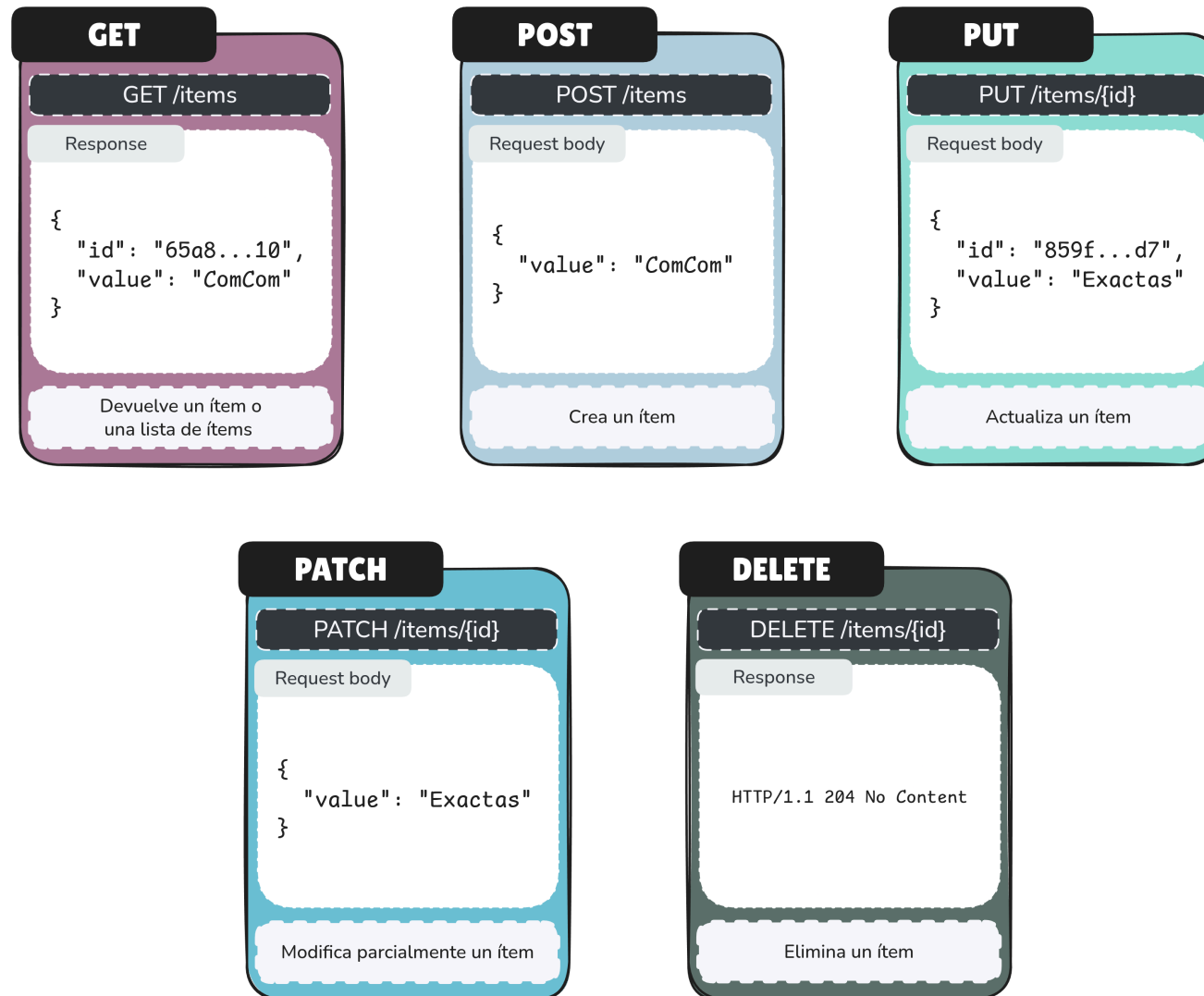
**forced to
shit out web apps**



Las APIs RESTful son un tipo de API que sigue los **principios de diseño de REST** (Representational State Transfer).

- Utilizan HTTP para realizar operaciones CRUD (Create, Read, Update, Delete) sobre recursos.
- **Un recurso es cualquier entidad que se pueda identificar, como un usuario, un producto, una publicación, etc.**
 - Se puede pensar *vagamente* como «información almacenada identificable».

Para entender como funcionan las APIs RESTful, es importante conocer los métodos HTTP más comunes:



¿Cómo devuelven la información?

Las APIs RESTful suelen devolver la información en formato JSON, aunque también pueden usar XML u otros formatos. A su vez, utilizan códigos de estado HTTP.

```
1 {  
2   "id": 1,  
3   "name": "Juan Pérez",  
4   "email": "jperez@dc.uba.ar",  
5   ...  
6 }
```

 JSON

HTTP/1.1 200 OK

Es importante remarcar que **no toda request devuelve contenido** en el cuerpo.

Por ejemplo, una request de eliminación exitosa del verbo DELETE suele devolver un código de estado 204 No Content, indicando que la operación fue exitosa pero no hay contenido para devolver.

¿Cómo devuelven la información?

Es importante remarcar que **no toda request devuelve contenido** en el cuerpo.

Por ejemplo, una request de eliminación exitosa del verbo DELETE suele devolver un código de estado 204 No Content, indicando que la operación fue exitosa pero no hay contenido para devolver.

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 50

{
  "status": 500,
  "message": "Internal Server Error"
}
```



También está bueno **ser consistentes** con los códigos de error y **no devolver mensajes redundantes**.

Los códigos de estado HTTP son respuestas estándar que indican el resultado de la solicitud realizada a la API.

- 1xx: Informativos
- 2xx: Éxito
- 3xx: Redirección
- 4xx: Error del cliente
- 5xx: Error del servidor

HTTP status ranges in a nutshell:

1xx: hold on

2xx: here you go

3xx: go away

4xx: you fucked up

5xx: I fucked up

-via @abt_programming

Los headers (encabezados) son **metadatos** que viajan junto a la request (cliente → servidor) o la response (servidor → cliente).

Contienen información adicional que no va en el body.

Los headers (encabezados) son **metadatos** que viajan junto a la request (cliente → servidor) o la response (servidor → cliente).

Contienen información adicional que no va en el body.

```
1 GET /me HTTP/1.1
2 Host: api.ejemplo.com
3 Authorization: Bearer eyJhbGciOiJIUz...
4 Content-Type: application/json
```

Los headers (encabezados) son **metadatos** que viajan junto a la request (cliente → servidor) o la response (servidor → cliente).

Contienen información adicional que no va en el body.

```
1 GET /me HTTP/1.1
2 Host: api.ejemplo.com
3 Authorization: Bearer eyJhbGciOiJIUz...
4 Content-Type: application/json
```

Algunos headers comunes:

- Content-Type: indica el formato del body (p. ej. application/json).
- Authorization: información de autenticación y autorización.

Hay varias formas de interactuar con una API RESTful, algunas de las más comunes son:

- Desde la terminal usando herramientas como `curl` o `httpie`.
- Usando clientes HTTP como Postman o Insomnia.
- Usando la propia documentación de la API, si esta tiene una interfaz interactiva (como Swagger UI o Scalar).

```
λ ~/ curl http://localhost:3000/items/ \
  --header 'Accept: application/json, multipart/form-data, text/plain'
{"id":"dafe32f6-fcd3-45d3-9fec-ca7ae0fcdee9","value":"ComCom"}%
```

```
λ ~/ http GET http://localhost:3000/items/ \
  Accept:'application/json, multipart/form-data, text/plain'
HTTP/1.1 200 OK
Content-Length: 62
Content-Type: application/json; charset=utf-8
Date: Tue, 21 Apr 2026 17:49:55 GMT

{
  "id": "dafe32f6-fcd3-45d3-9fec-ca7ae0fcdee9",
  "value": "ComCom"
}
```

Cientes HTTP (p. ej.: Postman)

The screenshot displays the Postman web interface. On the left sidebar, there are sections for 'COLLECTIONS' (containing 'My Collection'), 'ENVIRONMENTS', 'SPECS', and 'FLOWS'. The main workspace is titled 'Overview' and shows a GET request to 'http://localhost:3000/items/'. The 'Params' tab is active, showing a table for query parameters. Below this, the 'Body' tab is selected, displaying a JSON response. The response status is '200 OK' with a response time of 72 ms and a size of 184 B. The JSON body contains an object with 'id' and 'value' fields.

Overview GET http://localhost:3000/items/ No environment

Save Share

GET http://localhost:3000/items/ Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 3 Test Results

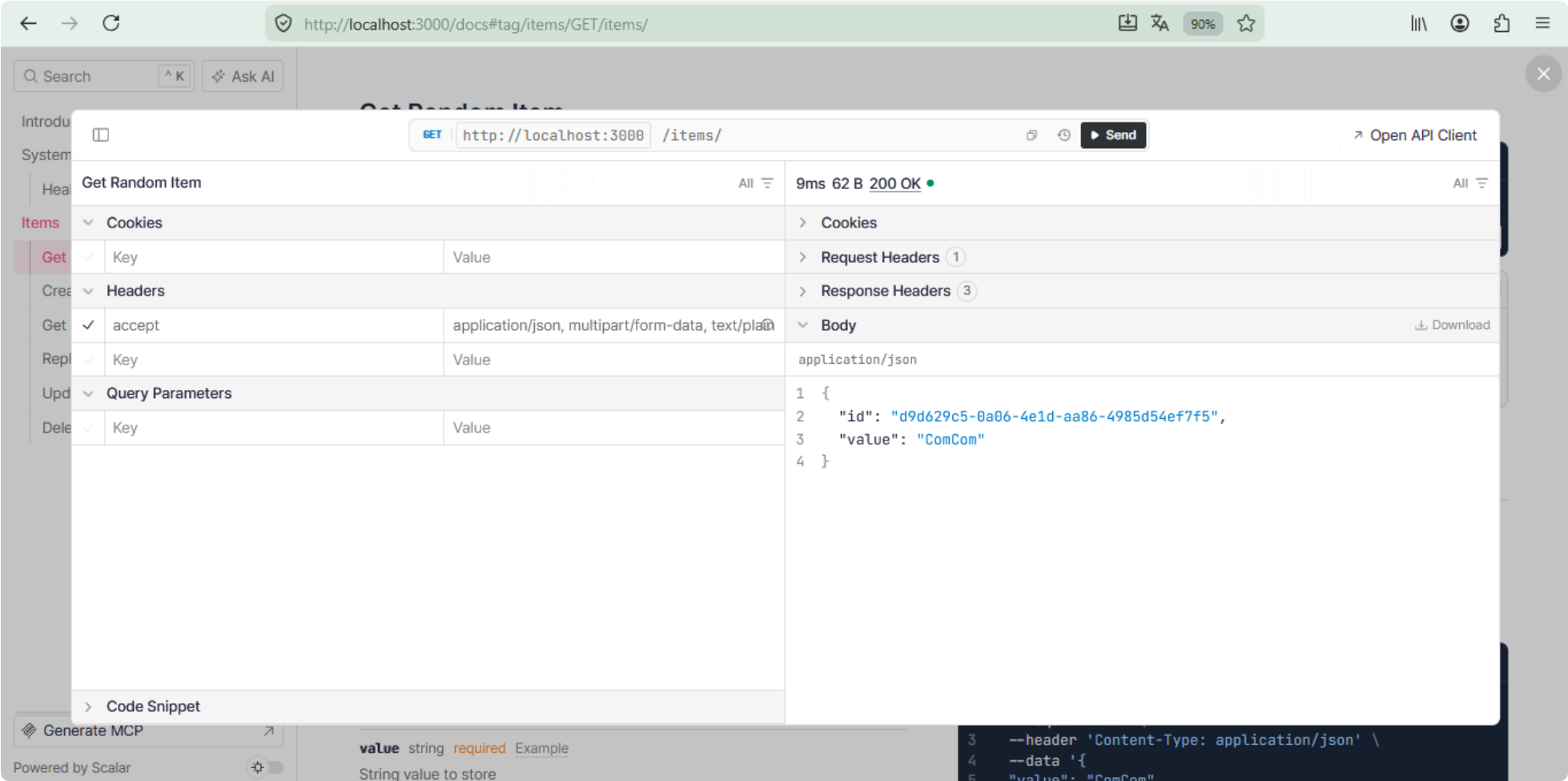
200 OK • 72 ms • 184 B •

{ } JSON Preview Visualize

```
1 {
2   "id": "8b952ff6-e6c6-4b62-9f4b-fc53bdb49059",
3   "value": "ComCom"
4 }
```

Connect Git Console Terminal Globals Vault Tools

Documentación interactiva (p. ej.: Scalar)



Interactuando con una API RESTful

Levantamos un servidor, este expone una API RESTful lista para usar. La idea de este ejercicio es familiarizarse con alguna forma para interactuar con la API (terminal, cliente HTTP o documentación interactiva) y probar los distintos endpoints que expone la API.

Levantamos una API RESTful en **taller-apis.cubawiki.com.ar**, esta API expone:

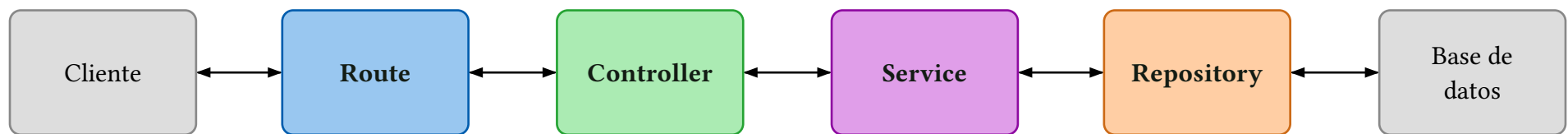
- `/items`: permite solicitudes GET y POST.
- `/items/{id}`: permite solicitudes GET, PUT, PATCH y DELETE.

Consignas:

- Entren a **/docs** de la API, donde van a encontrar la documentación interactiva.
- Prueben los distintos verbos HTTP sobre las rutas indicadas y observen las respuestas que devuelve la API.¹

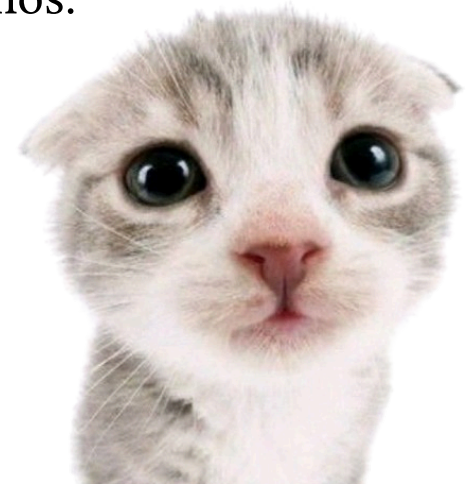
¹En el taller usaremos la documentación interactiva, pero pueden usar cURL o lo que prefieran.

¿Cómo se estructura un proyecto?



- La **route** recibe la request y la dirige al **controller**.
- El **controller** decide qué hacer y delega al **service**.
- El **service** ejecuta la lógica de negocio y usa el **repository** para acceder a los datos.
- Los **schemas** validan los datos automáticamente (los vamos a ver más adelante).

Vamos a ir viendo cada una de estas partes a medida que las necesitemos.



Una forma de organizar el código es separarlo en directorios, cada uno con una responsabilidad:

1	src/	
2	├─ routes/	Definen los endpoints (paths + verbos HTTP)
3	├─ controllers/	Reciben la request y deciden la respuesta
4	├─ services/	Contienen la lógica de negocio
5	├─ repositories/	Acceso a la base de datos
6	├─ schemas/	Definen la forma (y validación) de los datos
7	├─ middlewares/	Funciones que interceptan requests
8	└─ databases/	Configuración/conexión a la base de datos

También puede estar presente un directorio `utils` para funciones de utilidad.

Generalmente, para cada recurso (p. ej. `items`, `users`, `auth`) existe un archivo en `routes`, `controllers` y `services`; a veces también en `repositories`.

Dependiendo del proyecto, las tecnologías utilizadas o las preferencias del equipo, algunos directorios pueden no existir o pueden tener otros nombres.

Rutas / Routes / Endpoints

Una ruta (también conocida como endpoint) es una URL que representa un recurso o una colección de recursos en una API RESTful.

- Las rutas se utilizan para acceder a los recursos y realizar operaciones sobre ellos utilizando los verbos HTTP.

En el ejercicio de los strings anónimos, las rutas expuestas por la API RESTful son:

En el ejercicio de los strings anónimos, las rutas expuestas por la API RESTful son:

- `/items`: representa la colección de ítems y permite realizar operaciones como obtener un ítem aleatorio de la lista (GET) o crear un nuevo ítem (POST).

En el ejercicio de los strings anónimos, las rutas expuestas por la API RESTful son:

- `/items`: representa la colección de ítems y permite realizar operaciones como obtener un ítem aleatorio de la lista (GET) o crear un nuevo ítem (POST).
- `/items/{id}`: representa un ítem específico identificado por su `id` y permite realizar operaciones como obtener los detalles de un ítem (GET), actualizar un ítem (PUT o PATCH) o eliminar un ítem (DELETE).

- Route parameters (p. ej., {id}): se utilizan para identificar recursos específicos.
 - `/users/:id`
- Query parameters: se pueden utilizar, por ejemplo, para filtrar, ordenar o paginar recursos.
 - `/users?age=30&sort=asc&page=2`
- Body: se utiliza para enviar datos al servidor en solicitudes POST, PUT o PATCH.
 - `{ "name": "Juan Pérez" }`
- Response: es la información que el servidor devuelve al cliente después de procesar una solicitud.
 - `{ "id": 1, "name": "Juan Pérez" }`
 - `HTTP/1.1 200 OK`

Ejercicio 2: Creación de rutas (para Colapinto)

En el template tienen una API con varias rutas ya definidas, por ejemplo:

```
1 export const ejemploRoutes = new Elysia()  
2   .get("/ejemploGet", ejemploController.ejemploRecursoGet)  
3   .post("/ejemploPost", ejemploController.ejemploRecursoPost);
```

TS TypeScript

Consigna:

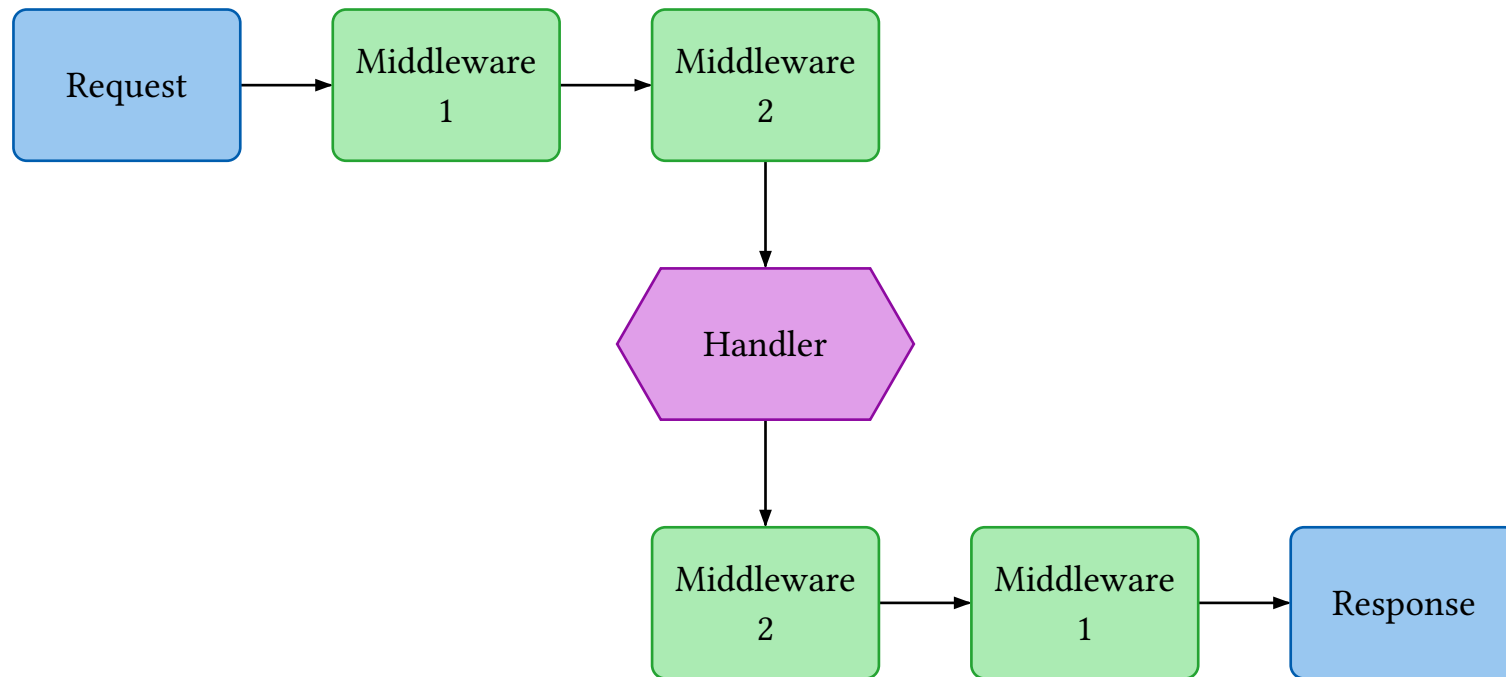
1. Descárguense en template y ábralo en su editor de código.
2. Hagan **bun install** para instalar las dependencias.
3. Abran **auth.routes.ts** ubicado en la carpeta `src/routes`.
4. Registren dos rutas de tipo POST:
 - `/auth/register` → `authController.register`
 - `/auth/login` → `authController.login`
5. Hagan **bun dev** para levantar la API.
6. Verifiquen que las rutas aparezcan en **`http://localhost:3000/docs`**.

Middlewares

¿Qué es un middleware?

Un middleware es una función que se ejecuta **antes** y/o **después** de que un handler (controller) procese una request.

Se puede pensar como una «capa» que intercepta las solicitudes y respuestas.



Los middlewares permiten agregar funcionalidad a la aplicación sin modificar los handlers individualmente. Por ejemplo:

- **Logging:** registrar cada solicitud que llega al servidor.
- **Autenticación:** verificar que el usuario tenga un token válido.
- **Autorización:** verificar que el usuario tenga permiso para acceder a la ruta.
- **Validación:** verificar que los datos de la solicitud sean correctos.

```
1  import { Elysia } from 'elysia'
2
3  export function middleware_ejemplo(app: Elysia) {
4    let middleware_app = app
5      .onBeforeHandle(({ request, set }) => {
6        // Se ejecuta antes de llegar al handler
7      })
8      .onAfterHandle(({ request, set }) => {
9        // Se ejecuta después de llegar al handler
10     });
11
12    return middleware_app;
13 }
```

TsTypeScript

Hay más métodos que `.onBeforeHandle` y `.onAfterHandle`, por mencionar uno: `.derive` permite agregar contexto a la request (como por ejemplo, el tiempo exacto en que el cliente mandó la request).

¿Cómo se usa un middleware en Elysia?

Para usar un middleware, simplemente se agrega con `.use()`:

```
1  import { Elysia } from "elysia";  
2  import { logger } from "../middlewares/logger";  
3  import { authenticator } from "../middlewares/authenticator";  
4  
5  const app = new Elysia()  
6    .get("/sin-middleware", () => "No uso ningún middleware.")  
7    .use(logger)  
8    .get("/con-logger", () => "Uso logger.")  
9    .use(authenticator)  
10   .get("/secret", () => "Uso logger y luego authenticator.");
```

tsTypeScript

En el template tienen las rutas `/secret` y `/users` sin protección...

Consigna:

1. Accedan a `http://localhost:3000/docs`.
2. Mándenle un GET a `/secret` y `/users`, vean qué devuelven.
3. Abran los archivos `secret.routes.ts` y `users.routes.ts` ubicados en la carpeta `src/routes`.
4. Importen el middleware `authenticator` de `../middlewares/authenticator`.
5. Agreguen `.use(authenticator)` antes de la definición de la ruta.
6. Prueben acceder a `/secret` y `/users`: deberían recibir un status code: 400 y en el body: Falta el token de autenticación.

Autenticación

Las APIs RESTful son **stateless**: no recuerdan quién sos entre request y request.



Para resolver esto, se usa un esquema de **tokens**:

1. De una forma u otra, obtenés un **token de acceso** (access token).
2. Luego, en cada request que requiera autenticación, el cliente envía ese token en el header Authorization:
 - Authorization: Bearer <token>

Para resolver esto, se usa un esquema de **tokens**:

1. De una forma u otra, obtenés un **token de acceso** (access token).
2. Luego, en cada request que requiera autenticación, el cliente envía ese token en el header Authorization:
 - Authorization: Bearer <token>

En nuestra API el **token de acceso** se obtiene mediante el endpoint POST /auth/login.

1. El cliente envía sus credenciales (usuario y clave) al endpoint de login.
2. Si son correctas, el servidor devuelve un **token de acceso**.

El tipo de token más común es el **JWT** (JSON Web Token)¹.

Lo único que interesa para este taller, es que el token es un string que le permite al servidor identificar al usuario, garantizando que no fue alterado.

¹¡A pesar de la creencia popular, no es obligatorio usar JWT!

Las contraseñas **nunca se guardan en texto plano**. Se guardan como un **hash**: el resultado de una función de una sola vía.

Texto plano (MAL)

```
1 usuario: valen
2 clave: 123456
```

Hash (BIEN)

```
1 usuario: juani
2 clave: $2b$10$K4f3...
```

Las contraseñas **nunca se guardan en texto plano**. Se guardan como un **hash**: el resultado de una función de una sola vía.

Texto plano (MAL)

```
1 usuario: valen
2 clave: 123456
```

Hash (BIEN)

```
1 usuario: juani
2 clave: $2b$10$K4f3...
```

- Al **registrar**: se hashea la clave antes de guardarla.
- Al **hacer login**: se hashea la clave ingresada y se compara con el hash guardado.
- Si alguien accede a la base de datos, **no puede recuperar las claves originales**.

Controllers, Services y Repositories

¿Qué es un Controller?

Un controller **recibe la request** HTTP, llama al service correspondiente, y **decide qué responder** (código de estado + body).

```
1 export function tweetAnónController(contexto) {  
2     if (!contexto.body || !contexto.body.textoTweet) {  
3         set.status = 400;  
4         return "Falta el body o el texto del tweet";  
5     }  
6     return tweetAnónService.publicarTweetAnón(contexto.uuid,  
7         contexto.body.textoTweet);  
}
```

tsTypeScript

¿Qué es un Controller?

Un controller **recibe la request** HTTP, llama al service correspondiente, y **decide qué responder** (código de estado + body).

```
1 export function tweetAnónController(contexto) {  
2     if (!contexto.body || !contexto.body.textoTweet) {  
3         set.status = 400;  
4         return "Falta el body o el texto del tweet";  
5     }  
6     return tweetAnónService.publicarTweetAnón(contexto.uuid,  
7         contexto.body.textoTweet);  
}
```

tsTypeScript

- **No contiene lógica de negocio** (eso va en el service).
- Se encarga de interpretar el resultado y elegir el **status code** adecuado.
- Maneja los **errores** que puede lanzar el service.

¿Qué es un Service?

Un service contiene la **lógica de negocio**: aplica reglas, valida campos según las reglas de negocio y usa el **repository** para acceder a los datos.

```
1 export function publicarTweetAnón(uuid, textoTweet) {  
2     const user = usersRepository.obtenerPorUuid(uuid);  
3     if (!user) throw new Error("Usuario no encontrado");  
4     if (user.baneado) throw new Error("Usuario baneado");  
5     if (textoTweet.length > 280) throw new Error("El texto del tweet es  
demasiado largo");  
6     const tweet = tweetAnónRepository.crear(uuid, textoTweet);  
7     return tweet;  
8 }
```

TS TypeScript

¿Qué es un Service?

Un service contiene la **lógica de negocio**: aplica reglas, valida campos según las reglas de negocio y usa el **repository** para acceder a los datos.

```
1 export function publicarTweetAnón(uuid, textoTweet) {  
2     const user = usersRepository.obtenerPorUuid(uuid);  
3     if (!user) throw new Error("Usuario no encontrado");  
4     if (user.baneado) throw new Error("Usuario baneado");  
5     if (textoTweet.length > 280) throw new Error("El texto del tweet es  
demasiado largo");  
6     const tweet = tweetAnónRepository.crear(uuid, textoTweet);  
7     return tweet;  
8 }
```

TS TypeScript

- **No conoce HTTP**: no sabe de requests, responses, ni status codes.
- **No accede a la base de datos directamente**: usa el repository.
- Lanza **errores** que el controller traduce a respuestas HTTP.

¿Qué es un Repository?

Un repository se encarga del **acceso a los datos**: leer, escribir, actualizar y eliminar registros de la base de datos.

■ `usersRepository.obtenerPorUuid(uuid):`

```
1 export function obtenerPorUuid(uuid) {  
2     // Código que obtiene el usuario de la base de datos usando su UUID  
3 }
```

TS TypeScript

■ `tweetAnónRepository.crear(uuid, textoTweet):`

```
1 export function crear(uuid, textoTweet) {  
2     // Código que crea un nuevo tweet en la base de datos con el UUID del  
    usuario y el texto del tweet  
3 }
```

TS TypeScript

¿Qué es un Repository?

Un repository se encarga del **acceso a los datos**: leer, escribir, actualizar y eliminar registros de la base de datos.

- `usersRepository.obtenerPorUuid(uuid):`

```
1 export function obtenerPorUuid(uuid) {  
2     // Código que obtiene el usuario de la base de datos usando su UUID  
3 }
```

TS TypeScript

- `tweetAnónRepository.crear(uuid, textoTweet):`

```
1 export function crear(uuid, textoTweet) {  
2     // Código que crea un nuevo tweet en la base de datos con el UUID del  
    usuario y el texto del tweet  
3 }
```

TS TypeScript

- Es el **único** que habla con la base de datos.
- Expone operaciones **CRUD** (Create, Read, Update, Delete).
- No tiene lógica de negocio ni conoce HTTP.

	Controller	Service	Repository
¿Conoce HTTP?	✓	×	×
¿Lógica de negocio?	×	✓	×
¿Accede a la DB?	×	×	✓
¿Decide status code?	✓	×	×
¿Valida datos?	Puede ¹	Reglas de negocio	×

¹Lo ideal es que no tenga que hacerlo o lo haga lo menos posible, ya veremos por qué.

Algunas operaciones **tardan**: leer un archivo, hacer una consulta a la DB, llamar a otra API...

Algunas operaciones **tardan**: leer un archivo, hacer una consulta a la DB, llamar a otra API...

TypeScript/JavaScript **no se queda esperando**. En vez de eso, te devuelve una **promesa**: un objeto que dice «*todavía no tengo el resultado, pero te lo voy a dar cuando esté listo*».

```
1 // Esto NO devuelve el usuario, devuelve una PROMESA de un
  usuario.
2 const promesa = db.query("SELECT * FROM users WHERE uuid = ?", [uuid]);
```



Algunas operaciones **tardan**: leer un archivo, hacer una consulta a la DB, llamar a otra API...

TypeScript/JavaScript **no se queda esperando**. En vez de eso, te devuelve una **promesa**: un objeto que dice «*todavía no tengo el resultado, pero te lo voy a dar cuando esté listo*».

```
1 // Esto NO devuelve el usuario, devuelve una PROMESA de un usuario.
2 const promesa = db.query("SELECT * FROM users WHERE uuid = ?", [uuid]);
```



Para decirle a TypeScript/JavaScript «*esperá a que termine antes de seguir*», usamos `await`:

```
1 // Ahora sí, esperamos a que la promesa se resuelva y obtenemos el usuario.
2 const usuario = await db.query("SELECT * FROM users WHERE uuid = ?", [uuid]);
```



Para poder usar `await` dentro de una función, hay que marcarla como `async`:

```
1  async function obtenerUsuario(uuid) {  
2    const usuario = await db.query("SELECT * FROM users WHERE uuid = ?",  
    [uuid]);  
3    return usuario;  
4  }
```

TypeScript

- Si una función es `async`, **siempre devuelve una promesa** (aunque parezca que devuelve un valor directo).
- Si se olvidan de poner `await`, van a recibir el objeto `Promise` en vez del resultado.

Parte A – Service (`auth.service.ts`):

1. Implementen registrar: hashen la clave (con `await hashearClave(clave)`) y creen el usuario (con `crearUsuario(nombreDeUsuario, claveHasheada)`), luego devuelvan su UUID.
2. Implementen login: obtengan el objeto usuario (con `obtenerUsuarioPorNombreDeUsuario(nombreDeUsuario)`) y comparen la clave que les pasaron con la clave guardada (con `await compararClaves(clave, usuario.claveHasheada)`), luego devuelvan su UUID.

Parte B – Controller (`auth.controller.ts`):

1. Completen registrar: validen que el body exista, luego sus parámetros (que existan, largos mínimos), llamen al service dentro del try/catch y devuelvan el token con el status code 201 o que el usuario ya existe con el status code 409, según corresponda.
2. Completen login: validen que el body exista, luego sus parámetros (que existan), llamen al service dentro del try/catch y devuelvan el token con el status code 200 o que las credenciales son inválidas con el status code 401, según corresponda.

Parte A – Service (`auth.service.ts`):

1. Implementen registrar: hashen la clave (con `await hashearClave(clave)`) y creen el usuario (con `crearUsuario(nombreDeUsuario, claveHasheada)`), luego devuelvan su UUID.
2. Implementen login: obtengan el objeto usuario (con `obtenerUsuarioPorNombreDeUsuario(nombreDeUsuario)`) y comparen la clave que les pasaron con la clave guardada (con `await compararClaves(clave, usuario.claveHasheada)`), luego devuelvan su UUID.

Parte B – Controller (`auth.controller.ts`):

1. Completen registrar: validen que el body exista, luego sus parámetros (que existan, largos mínimos), llamen al service dentro del try/catch y devuelvan el token con el status code 201 o que el usuario ya existe con el status code 409, según corresponda.
2. Completen login: validen que el body exista, luego sus parámetros (que existan), llamen al service dentro del try/catch y devuelvan el token con el status code 200 o que las credenciales son inválidas con el status code 401, según corresponda.

Prueben: registrar un usuario, hacer login, intentar con datos inválidos.

¿Cuántos `if` tuvieron que escribir para validar los datos?

¿Qué pasa si agregan un campo nuevo al body? ¿Y si cambian un largo mínimo?

Schemas

¿Qué es un Schema?

Un schema **define la forma de los datos**: qué campos tiene el body, qué tipos tienen, qué restricciones cumplen, y qué devuelve la respuesta.

```
1  const bodySchema = t.Object({  
2    nombreDeUsuario: t.String({ minLength: 3 }),  
3    clave: t.String({ minLength: 6 }),  
4  });
```

ts TypeScript

¿Qué es un Schema?

Un schema **define la forma de los datos**: qué campos tiene el body, qué tipos tienen, qué restricciones cumplen, y qué devuelve la respuesta.

```
1  const bodySchema = t.Object({  
2    nombreDeUsuario: t.String({ minLength: 3 }),  
3    clave: t.String({ minLength: 6 }),  
4  });
```

ts TypeScript

Si la request no cumple con el schema, **el framework la rechaza automáticamente** con un 400 Bad Request, sin necesidad de escribir un solo if.

Sin schema (register en auth.controller.ts), no entra todo el código en la diapositiva:

```
1  // ... código anterior
2  if (!body) {
3    set.status = 400
4    return "Falta body en la request"
5  }
6  const { nombreDeUsuario, clave } = body;
7  if (!nombreDeUsuario || !clave) {
8    set.status = 400
9    return "Faltan el nombre del usuario y la clave en la request";
10 }
11 if (nombreDeUsuario.length < 3) {
12   set.status = 400
13   return "El nombre de usuario debe tener al menos 3 caracteres";
14 }
15 // ... una validación similar para la clave y el resto del código
```

TS TypeScript

Con schema (register en auth.controller.ts), entra todo el código en la diapositiva:

```
1  const { body, set } = contexto;
2  const { nombreDeUsuario, clave } = body;
3
4  // Acá estarían los chequeos manuales, pero ya no hacen falta porque
   el schema se encarga de eso.
5
6  try {
7      const uuid = await authService.register(nombreDeUsuario, clave)
8
9      const tokenDeAcceso = await crearTokenDeAcceso(uuid)
10     set.status = 201
11     return {tokenDeAcceso}
12 } catch {
13     set.status = 409
14     return "Ya existe un usuario con ese nombre de usuario";
15 }
```

TS TypeScript

¿Cómo se usa un Schema?

El schema se pasa como **tercer argumento** en la definición de la ruta:

```
1 // Antes (sin schema):  
2 .post("/auth/register", authController.register)  
3  
4 // Después (con schema):  
5 .post("/auth/register", authController.register,  
  registrarUnUsuarioSchema)
```

TS TypeScript

¿Cómo se usa un Schema?

El schema se pasa como **tercer argumento** en la definición de la ruta:

```
1 // Antes (sin schema):  
2 .post("/auth/register", authController.register)  
3  
4 // Después (con schema):  
5 .post("/auth/register", authController.register,  
  registrarUnUsuarioSchema)
```

TS TypeScript

Además, el controller puede tipar sus parámetros con el **contrato** del schema:

```
1 export async function register(  
2   contexto: Context<RegistrarUnUsuarioRouteContract> // Toma un  
   contexto que cumple con el contrato  
3 ): Promise<RegistrarUnUsuarioRouteContract['response'][201 | 409]> //  
   Devuelve 201 o 409, según el contrato  
4 { ... }
```

TS TypeScript

Esto da **autocompletado** y **chequeo de tipos** en el editor.

Al definir schemas, la documentación interactiva (Swagger / Scalar) muestra **automáticamente**:

- Los campos esperados en el body, con sus tipos y restricciones.
- Las posibles respuestas con sus códigos de estado.
- Ejemplos de uso.

Al definir schemas, la documentación interactiva (Swagger / Scalar) muestra **automáticamente**:

- Los campos esperados en el body, con sus tipos y restricciones.
- Las posibles respuestas con sus códigos de estado.
- Ejemplos de uso.

Todo sin escribir documentación a mano. Solo por definir los schemas.

Parte A – Schemas (`auth.schemas.ts`):

1. **Completen el schema** `const` `iniciarSesionSchema` = { ... }¹:
 - Body: `nombreDeUsuario` (string), `clave` (string).
 - Response 200: objeto con `tokenDeAcceso` (string).
2. **Completen el contrato** `interface` `IniciarSesionRouteContract` { ... }²:
 - Body: `nombreDeUsuario` (string), `clave` (string).
 - Response 200: objeto con `tokenDeAcceso` (string).
 - Response 401: string (mensaje de error).
3. Modifiquen `auth.routes.ts`: pasen los schemas como tercer argumento a cada `.post()`.
 - `.post("/auth/register", authController.register, registrarUnUsuarioSchema)`
 - `.post("/auth/login", authController.login, iniciarSesionSchema)`

¹Básense en el schema `const` `registrarUnUsuarioSchema` = { ... } que ya está implementado.

²El contrato `interface` `RegistrarUnUsuarioRouteContract` { ... } ya está implementado, úsenlo como referencia.

Parte B – Autocompletado y chequeo de tipos (opcional):

1. Cambien la signatura de las funciones en `auth.controller.ts` para que usen los contratos de los schemas.

```
■ export async function register(contexto:
  Context<RegistrarUnUsuarioRouteContract>):
  Promise<RegistrarUnUsuarioRouteContract['response'] [201 | 409]> { ... }
```

```
■ export async function login(contexto:
  Context<IniciarSesionRouteContract>):
  Promise<IniciarSesionRouteContract['response'] [200 | 401]> { ... }
```

2. **Eliminen las validaciones manuales** que ahora son redundantes.
3. Vayan a `/docs` y vean la documentación auto-generada.



¿¿¿¿¿¿ Preguntas??????

Tecnologías usadas en el taller

- Bun (runtime): <https://bun.sh/docs>
- Elysia (framework): <https://elysiajs.com/>
- TypeScript (lenguaje): <https://www.typescriptlang.org/docs/>

REST y HTTP

- El origen de los principios REST: Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*
- HTTP response status codes: [MDN Web Docs](#)
- HTTP request methods: [MDN Web Docs](#)
- ¿Qué es una API REST?: [Red Hat](#)

Autenticación y seguridad

- Introducción a JWT: <https://jwt.io/introduction>
- OWASP API Security Top 10: <https://owasp.org/API-Security/>
- Argon2 (hashing de contraseñas): [Wikipedia](#)

Herramientas

- Scalar (documentación interactiva): <https://scalar.com/>
- Postman: <https://www.postman.com/>
- curl: <https://curl.se/>
- httpie: <https://httpie.io/>

Les dejamos una lista de algunas APIs públicas por si les interesa investigar:

- APIs List: <https://apislist.com/>
- Free APIs: <https://free-apis.github.io/>
- Transporte Buenos Aires: <https://api-transporte.buenosaires.gob.ar/>
- MercadoPago: <https://www.mercadopago.com.ar/developers/es/reference>
- Spotify: <https://developer.spotify.com/documentation/web-api>

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine. https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf
- Fielding, R. T., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. RFC 7231. <https://datatracker.ietf.org/doc/html/rfc7231>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. <https://datatracker.ietf.org/doc/html/rfc7519>
- Mozilla Developer Network. *HTTP Reference*. <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- OWASP Foundation. *API Security Top 10*. <https://owasp.org/API-Security/>